

torch.linalg.vector_norm#

https://pytorch.org/docs/stable/generated/torch.linalg.vector_norm.html

Description:

Computes a vector norm. If x is complex valued, it computes the norm of $x.abs()$. Supports input of float, double, cfloat and cdouble dtypes. This function does not necessarily treat multidimensional x as a batch of vectors, instead: If $dim = None$, x will be flattened before the norm is computed.

Function Signature

```
torch.linalg.vector_norm(x, ord=2, dim=None, keepdim=False, *, dtype=None, out=None) → Tensor#
```

Parameters

Keyword Arguments

Returns

Parameters

Keyword

out (Tensor, optional) – output tensor. Ignored if None. Default: None. dtype (torch.dtype, optional) – type used to perform the accumulation and the return. If specified, x is cast to dtype before performing the operation, and the returned tensor's type will be dtype if real and of its real counterpart if complex. dtype may be complex if x is complex, otherwise it must be real. x should be convertible without narrowing to dtype. Default: None

Returns:

A real-valued tensor, even when x is complex.

Code Examples

```
>>> from torch import linalg as LA
>>> a = torch.arange(9, dtype=torch.float) - 4
>>> a
tensor([-4., -3., -2., -1.,  0.,  1.,  2.,  3.,  4.])
>>> B = a.reshape((3, 3))
>>> B
tensor([[ -4.,  -3.,  -2.],
        [ -1.,   0.,   1.],
        [  2.,   3.,   4.]])
>>> LA.vector_norm(a, ord=3.5)
tensor(5.4345)
>>> LA.vector_norm(B, ord=3.5)
tensor(5.4345)
```

Notes & Warnings

NOTE See also `torch.linalg.matrix_norm()` computes a matrix norm.

Detailed Documentation

Documentation

Computes a vector norm.

If x is complex valued, it computes the norm of $x.abs()$

Supports input of float, double, cfloat and cdouble dtypes.

This function does not necessarily treat multidimensional x as a batch of vectors, instead:

If $dim = None$, x will be flattened before the norm is computed.

If dim is an int or a tuple, the norm will be computed over these dimensions and the other dimensions will be treated as batch dimensions.

This behavior is for consistency with `torch.linalg.norm()`.

ord defines the vector norm that is computed. The following norms are supported:

$$\text{sum}(\text{abs}(x)^{\{ord\}})^{\{1 / ord\}}$$

where inf refers to `float('inf')`, NumPy's `inf` object, or any equivalent object.

$dtype$ may be used to perform the computation in a more precise dtype. It is semantically equivalent to calling `linalg.vector_norm(x.to(dtype))` but it is faster in some cases.

`torch.linalg.matrix_norm()` computes a matrix norm.

x (Tensor) – tensor, flattened by default, but this behavior can be controlled using dim .
(Note: the keyword argument `input` can also be used as an alias for x .)

ord (int, float, inf, -inf, 'fro', 'nuc', optional) – order of norm. Default: 2

dim (int, Tuple[int], optional) – dimensions over which to compute the norm. See above for the behavior when $dim = None$. Default: None

`keepdim` (bool, optional) – If set to `True`, the reduced dimensions are retained in the result as dimensions with size one. Default: `False`

`out` (Tensor, optional) – output tensor. Ignored if `None`. Default: `None`.

`dtype` (torch.dtype, optional) – type used to perform the accumulation and the return. If specified, `x` is cast to `dtype` before performing the operation, and the returned tensor's type will be `dtype` if real and of its real counterpart if complex. `dtype` may be complex if `x` is complex, otherwise it must be real. `x` should be convertible without narrowing to `dtype`. Default: `None`

A real-valued tensor, even when `x` is complex.